Programmes *analyse_partie_json_joueur.pl*, *analyses_parties_json_joueur.pl*, *analyses_parties_json_joueurs_dir.pl* (voir *rejoue_partie.pl*)

1 *analyse_partie_json_joueur.pl*

A quoi sert il ?

Il analyse une partie déjà jouée (format *json*) et écrit un fichier (format *prolog* et texte) contenant les paramètres de la partie, les coups joués, les réponses du système, et pour chaque coup les explications des réductions des variables.

Où est il ?

Dans le dossier *programs*.

Comment l'appeler ?

```
swipl -s analyse_partie_json_joueur.pl
      -g  "analyse_partie(
                <path of the file containing the played game>,  (input, String)
                <path of the output file>,                      (input, String)
                <player's name>,                                (input, String)
                <arbitrary number of the game>                  (input, Integer)
            )"
```

A noter que le nombre associé à une partie est arbitrairement fourni, il peut ne pas correspondre au numéro contenu dans le nom du fichier json.

Exemple d'appel :

```
swipl -s analyse_partie_joueur.pl
      -g  "top(
               '../parties_joueur/colin/_partie_3.json',
               '../parties_joueur/colin/_partie_3.pl',
               colin,
               11
            )"
```

Format du fichier de sortie

Exemple du début de fichier de sortie :

```
ident(11,'20yellow=red<blue<green0yellow31green102blue43red3').
joueur(11,colin).
partie(11,'../parties_joueur/colin/_partie_3').
somme(11,20).
ordre_couleurs(11,[yellow,=,red,<,blue,<,green]).
solution(11,[c(0,yellow,3),c(1,green,10),c(2,blue,4),c(3,red,3)]).
nb_solutions(11,168).
cartes_jouees(11,
   [
   coup(1,168,6,[c(0,red,3,12.88,non,oui),c(1,blue,7,3.51,non,plus),
                 c(2,green,8,2.08,non,moins), c(3,yellow,2,1.82,non,plus)]),
     coup(2,6,2,[c(0,yellow,3,13.57,oui,oui),c(1,green,8,15.91,oui,plus),
                 c(2,blue,6,2.1,oui,moins),c(3,red,3,2.8,oui,oui)]),
     coup(3,2,1,[c(1,green,9,12.74,oui,plus),c(2,blue,5,1.47,oui,moins)]),
     coup(4,1,1,[c(1,green,10,29.37,oui,oui),c(2,blue,4,1.62,oui,oui)])
   ]
            ).
resultat(11,gagne).
```

Noter que 11 est le numéro arbitraire donnée pour cette partie, toutes les informations dans le fichier de sortie sont préfixées par ce numéro. Nous expliquons ci-après chaque ligne du fichier de sortie :

1. identifiant interne de la partie
2. nom du joueur,
3. localisation du fichier contenant la partie,
4. somme des valeurs,
5. ordre des couleurs,
6. solution,

7. nombre de solutions possibles,
8. cartes jouées où

```
coup(1,168,6,
      [
       c(0,red,3,12.88,non,oui), c(1,blue,7,3.51,non,plus),
       c(2,green,8,2.08,non,moins), c(3,yellow,2,1.82,non,plus)
      ]
     ),
```

signifie qu'au coup 1 il y avait 168 solutions avant le coup et 6 après le coup joué et la réponse du système, suivi des cartes jouées où

```
c(1, blue, 7, 3.51, non, plus),
```

signifie qu'en position 1 le joueur a proposé la couleur *blue* et la valeur 7, qu'il l'a fait 3.51 secondes après avoir joué le coupé précédent, et qu'en réponse le système lui a signalé que la couleur en cette position n'était pas la couleur proposée (non), et que la valeur réelle en cette position était supérieure (plus) à la valeur proposée.

La suite du fichier fournit les explications des réductions trouvées par le solveur :

```
********** no reduction after move 0 **********

************* explainations of the reductions after move 1 *************

-------------------------------------------------------------

the colour (c0) of the card in position 0 is in [blue,yellow] :

7 < v1                                      : answer of the system
v1 in [1,2,3,4,5,6,7,8,9,10]                : known domain
     => v1 in [8,9,10]

v0 is 3                                     : answer of the system
[dans(v1,[8,9,10])]=> v0 < v1

c0 avant c1 <=> v0 < v1                     : intial constraint
[v0<v1]=> c0 avant c1

c0 \= red                                   : answer of the system
```

```
c0 in [blue,green,red,yellow]           : known domain
     => c0 in [blue,green,yellow]

blue \= c1                              : answer of the system
c1 in [blue,green,red,yellow]           : known domain
     => c1 in [green,red,yellow]

c0 \= c1                                : intial constraint
red avant green                         : intial constraint
yellow avant green                      : intial constraint
[avant(c0,c1)]
[dans(c0,[blue,green,yellow])]
[dans(c1,[green,red,yellow])]=> c0 in [blue,yellow]

------------------------------------------------------------

the value (v0) of the card in position 0 is 3 :

v0 is 3                                 : seen

...
```